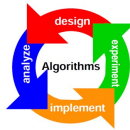


# Estrategias algorítmicas

José A. Rodríguez Melquiades

23 de mayo de 2026

# Estrategia algorítmica voraz



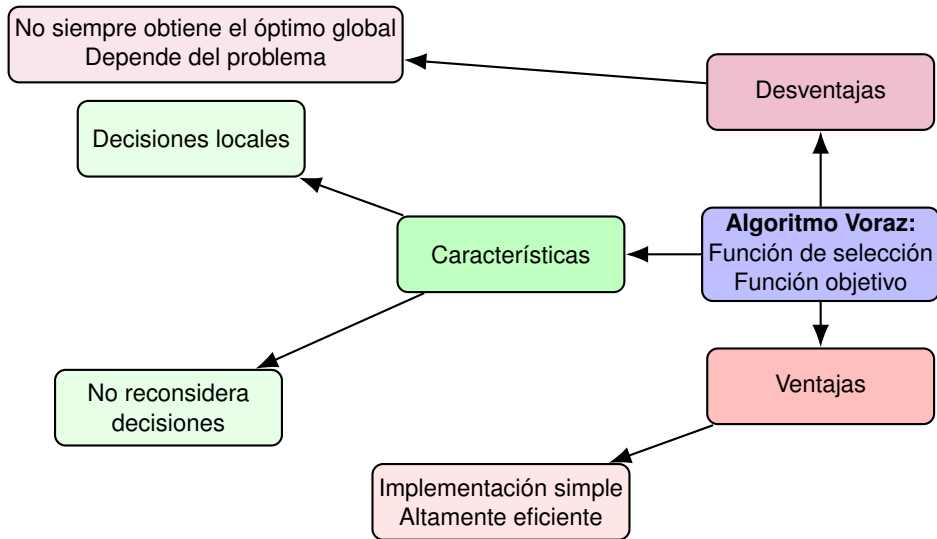
- 1 Algoritmo voraz
- 2 Relación entre algoritmo voraz y optimización
- 3 Problema de la mochila
  - Modelo de optimización: Mochila fraccional
  - Algoritmo voraz: Mochila fraccional
  - Modelo de optimización: Mochila binaria
  - Algoritmo voraz: Mochila binaria
  - Aplicaciones del problema de la mochila
- 4 Mochila para empaquetado inteligente

## Algoritmo voraz

- **Algoritmo voraz<sup>1</sup>** : Son técnicas de resolución de problemas que siguen un enfoque heurístico, donde su característica es tomar decisiones locales óptimas en cada paso con la esperanza de encontrar una solución global óptima para el problema general.

---

<sup>1</sup>Algorithm Greedy-AG



- Algoritmo voraz:

- **Algoritmo voraz:**

- ▶ **Idea:** Construir una solución a través de una sucesión de pasos, cada uno expandiendo una solución parcial hasta hallar la solución del problema (función objetivo).

- **Algoritmo voraz:**

- ▶ **Idea:** Construir una solución a través de una sucesión de pasos, cada uno expandiendo una solución parcial hasta hallar la solución del problema (función objetivo).
- ▶ **Proceso:** En cada paso la elección debe ser:

- **Algoritmo voraz:**

- ▶ **Idea:** Construir una solución a través de una sucesión de pasos, cada uno expandiendo una solución parcial hasta hallar la solución del problema (función objetivo).
- ▶ **Proceso:** En cada paso la elección debe ser:
  - ★ **Viable:** Tiene que cumplir con las condiciones del problema.

## ● Algoritmo voraz:

- ▶ **Idea:** Construir una solución a través de una sucesión de pasos, cada uno expandiendo una solución parcial hasta hallar la solución del problema (función objetivo).
- ▶ **Proceso:** En cada paso la elección debe ser:
  - ★ **Viable:** Tiene que cumplir con las condiciones del problema.
  - ★ **Localmente óptima:** De todas las elecciones viables se debe elegir la mejor.

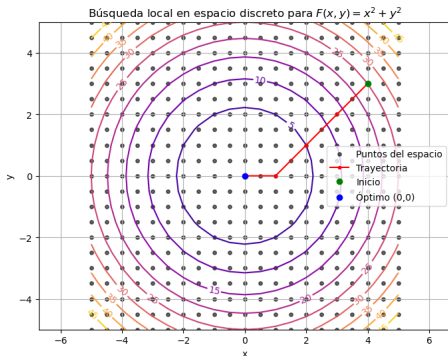
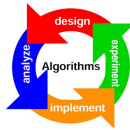


Figura 1: Mínimos locales y global.

## ● Seudocódigo:

- 1: **Input:**  $A, n$  ▷ Arreglo:  $A$ ; Tamaño:  $n$
- 2: **Output:** Solución óptima.
- 3:  $solucion := \phi$ ; ▷ Inicializa la solución
- 4: **for**  $i := 1$  to  $n$  **do**
- 5:      $x := select(A)$  ▷ Función select toma la entrada  $A$  y lo asigna a  $x$ .
- 6:     **if**  $viable(solucion, x)$  **then** ▷ Decide cuando incluir a  $x$  en la solución final.
- 7:          $solucion := Union(solucion, x)$
- 8:     **end if**
- 9: **end for**
- 10: Retorna  $solucion$

# Estrategia algorítmica voraz



- 1 Algoritmo voraz
- 2 Relación entre algoritmo voraz y optimización
- 3 Problema de la mochila
  - Modelo de optimización: Mochila fraccional
  - Algoritmo voraz: Mochila fraccional
  - Modelo de optimización: Mochila binaria
  - Algoritmo voraz: Mochila binaria
  - Aplicaciones del problema de la mochila
- 4 Mochila para empaquetado inteligente

# Relación entre algoritmo voraz y optimización

- **Función 1-dimensional:** Curva de  $F(x)$ .

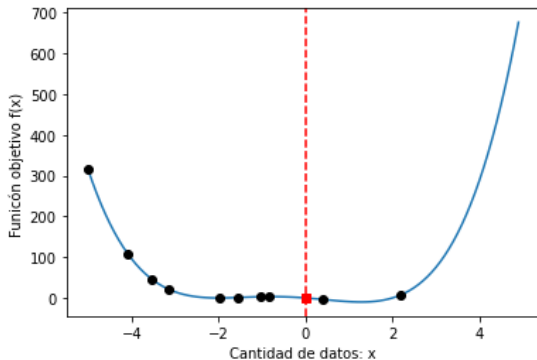


Figura 2: Linea de la función  $F(x) = (x - 2) \times x \times (x + 2)^2$ .

- **Función 2-dimensional:** Contorno de  $F(x, y)$ .

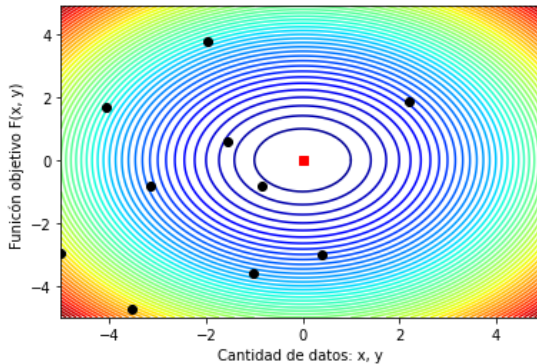


Figura 3: Contorno de la función  $F(x, y) = x^2 + y^2$ .

- **Función 3-dimensional:** Superficie de  $F(x, y)$ .

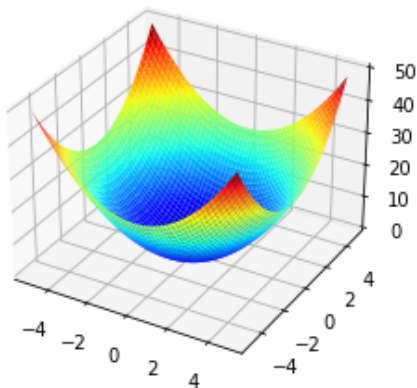


Figura 4: Superficie de la función  $f(x, y) = x^2 + y^2$ .

- **Objetivo:** Ambos buscan la mejor solución (óptima o casi óptima) para un problema, es decir, maximizar o minimizar.

- **Objetivo:** Ambos buscan la mejor solución (óptima o casi óptima) para un problema, es decir, maximizar o minimizar.
- **Tipo de problemas:** Los AG pueden ser usados para solucionar problemas de optimización combinatoria.

- **Objetivo:** Ambos buscan la mejor solución (óptima o casi óptima) para un problema, es decir, maximizar o minimizar.
- **Tipo de problemas:** Los AG pueden ser usados para solucionar problemas de optimización combinatoria.
- **Condiciones necesarias:** Para que un AG sea usado en optimización, el problema debe cumplir:

- **Objetivo:** Ambos buscan la mejor solución (óptima o casi óptima) para un problema, es decir, maximizar o minimizar.
- **Tipo de problemas:** Los AG pueden ser usados para solucionar problemas de optimización combinatoria.
- **Condiciones necesarias:** Para que un AG sea usado en optimización, el problema debe cumplir:
  - ▶ **Subestructura óptima:** La solución óptima global puede ser construida a partir de las soluciones óptimas de los subproblemas.

- **Objetivo:** Ambos buscan la mejor solución (óptima o casi óptima) para un problema, es decir, maximizar o minimizar.
- **Tipo de problemas:** Los AG pueden ser usados para solucionar problemas de optimización combinatoria.
- **Condiciones necesarias:** Para que un AG sea usado en optimización, el problema debe cumplir:
  - ▶ **Subestructura óptima:** La solución óptima global puede ser construida a partir de las soluciones óptimas de los subproblemas.
  - ▶ **Greedy choice property:** La elección localmente óptima conduce a una solución globalmente óptima.

- Aplicaciones:

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

- Aplicaciones:
  - ▶ Grafos

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

- Aplicaciones:

- ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

- Aplicaciones:

- ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
    - ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

- Aplicaciones:

- ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees

- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

- ▶ Planificación<sup>2</sup>

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

- Aplicaciones:

- ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
    - ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

- ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

## ● Aplicaciones:

### ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

### ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.
- ★ Planificación de trabajos prioritarios en un procesador - Scheduling of priority jobs on single processor.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

## ● Aplicaciones:

### ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

### ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.
- ★ Planificación de trabajos prioritarios en un procesador - Scheduling of priority jobs on single processor.

### ▶ Problemas de optimización combinatoria<sup>3</sup>

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

## ● Aplicaciones:

### ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

### ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.
- ★ Planificación de trabajos prioritarios en un procesador - Scheduling of priority jobs on single processor.

### ▶ Problemas de optimización combinatoria<sup>3</sup>

- ★ Problema de la mochila - Knapsack problem.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

## ● Aplicaciones:

### ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

### ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.
- ★ Planificación de trabajos prioritarios en un procesador - Scheduling of priority jobs on single processor.

### ▶ Problemas de optimización combinatoria<sup>3</sup>

- ★ Problema de la mochila - Knapsack problem.
- ★ Problema del agente viajero - Traveling Salesman problem.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

## ● Aplicaciones:

### ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

### ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.
- ★ Planificación de trabajos prioritarios en un procesador - Scheduling of priority jobs on single processor.

### ▶ Problemas de optimización combinatoria<sup>3</sup>

- ★ Problema de la mochila - Knapsack problem.
- ★ Problema del agente viajero - Traveling Salesman problem.
- ★ Problema de cubrimiento de conjuntos - Set-covering problem.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

## ● Aplicaciones:

### ▶ Grafos

- ★ Árbol generador mínimo - Minimum Spanning Trees
- ★ Ruta corta - Dijkstra's (shortest path) Algorithm.

### ▶ Planificación<sup>2</sup>

- ★ Selección de actividades - Activity Selection.
- ★ Planificación de trabajos prioritarios en un procesador - Scheduling of priority jobs on single processor.

### ▶ Problemas de optimización combinatoria<sup>3</sup>

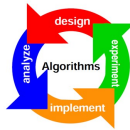
- ★ Problema de la mochila - Knapsack problem.
- ★ Problema del agente viajero - Traveling Salesman problem.
- ★ Problema de cubrimiento de conjuntos - Set-covering problem.
- ★ Problema de enpaquetamiento - Bin packing.

---

<sup>2</sup>Scheduling

<sup>3</sup>Combinatorial Optimization Problems

# Estrategia algorítmica voraz



- 1 Algoritmo voraz
- 2 Relación entre algoritmo voraz y optimización
- 3 Problema de la mochila
  - Modelo de optimización: Mochila fraccional
  - Algoritmo voraz: Mochila fraccional
  - Modelo de optimización: Mochila binaria
  - Algoritmo voraz: Mochila binaria
  - Aplicaciones del problema de la mochila
- 4 Mochila para empaquetado inteligente

## Problema de la mochila

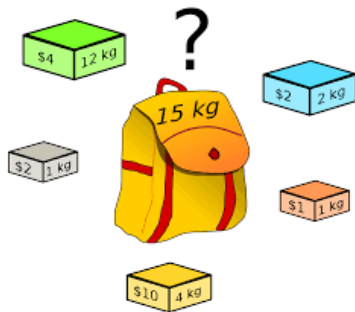


Figura 5: Mochila de capacidad 15 kg..

- Datos:

- Datos:
  - ▶ Objetos:  $n$ .

- Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .

## • Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- Problema:

## • Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

## • Problema:

- ▶ Si un objeto  $i$  es colocado en la mochila, entonces se obtendrá un beneficio o utilidad  $b_i x_i$ , es decir, determinar que objetos serán considerados para ser colocados en la mochila.

- Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- Problema:

- ▶ Si un objeto  $i$  es colocado en la mochila, entonces se obtendrá un beneficio o utilidad  $b_i x_i$ , es decir, determinar que objetos serán considerados para ser colocados en la mochila.

- Objetivo:

- Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- Problema:

- ▶ Si un objeto  $i$  es colocado en la mochila, entonces se obtendrá un beneficio o utilidad  $b_i x_i$ , es decir, determinar que objetos serán considerados para ser colocados en la mochila.

- Objetivo:

- ▶ Llenar la mochila de modo que se maximize el beneficio o utilidad sin vencer la capacidad de la mochila.

- Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- Problema:

- ▶ Si un objeto  $i$  es colocado en la mochila, entonces se obtendrá un beneficio o utilidad  $b_i x_i$ , es decir, determinar que objetos serán considerados para ser colocados en la mochila.

- Objetivo:

- ▶ Llenar la mochila de modo que se maximize el beneficio o utilidad sin vencer la capacidad de la mochila.

- Casos:

- Datos:

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- Problema:

- ▶ Si un objeto  $i$  es colocado en la mochila, entonces se obtendrá un beneficio o utilidad  $b_i x_i$ , es decir, determinar que objetos serán considerados para ser colocados en la mochila.

- Objetivo:

- ▶ Llenar la mochila de modo que se maximize el beneficio o utilidad sin vencer la capacidad de la mochila.

- Casos:

- ▶ Mochila fraccional.

- **Datos:**

- ▶ Objetos:  $n$ .
- ▶ Peso del objeto  $i$ :  $w_i, \forall i$ .
- ▶ Capacidad:  $m$ .

- **Problema:**

- ▶ Si un objeto  $i$  es colocado en la mochila, entonces se obtendrá un beneficio o utilidad  $b_i x_i$ , es decir, determinar que objetos serán considerados para ser colocados en la mochila.

- **Objetivo:**

- ▶ Llenar la mochila de modo que se maximize el beneficio o utilidad sin vencer la capacidad de la mochila.

- **Casos:**

- ▶ Mochila fraccional.
- ▶ Mochila binaria.

# Modelo de optimización: Mochila fraccional

- Modelo:

$$\text{MAX. } Z = \sum_{1 \leq i \leq n} b_i x_i \quad \text{Beneficio o utilidad} \quad (1)$$

s.a.

$$\sum_{1 \leq i \leq n} w_i x_i \leq m \quad \text{Llenar mochila según su capacidad } m \quad (2)$$

$$0 \leq x_i \leq 1, i = 1, \dots, n \quad \text{Fracción de peso} \quad (3)$$

- Ejemplo:

- Ejemplo:
  - ▶ Objetos:  $n = 3$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .
- ▶ Soluciones viables:

$$\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2\}$$

$$\sum \mathbf{w}_i \mathbf{x}_i$$

$$\sum \mathbf{b}_i \mathbf{x}_i$$

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .
- ▶ Soluciones viables:

| $\{x_0, x_1, x_2\}$                                      | $\sum w_i x_i$ | $\sum b_i x_i$ |
|----------------------------------------------------------|----------------|----------------|
| ★ $\left\{\frac{1}{2}, \frac{1}{3}, \frac{1}{4}\right\}$ | 16,5           | 24,25          |

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .
- ▶ Soluciones viables:

| $\{x_0, x_1, x_2\}$                                      | $\sum w_i x_i$ | $\sum b_i x_i$ |
|----------------------------------------------------------|----------------|----------------|
| ★ $\left\{\frac{1}{2}, \frac{1}{3}, \frac{1}{4}\right\}$ | 16,5           | 24,25          |
| ★ $\left\{1, \frac{2}{15}, 0\right\}$                    | 20             | 28,2           |

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .
- ▶ Soluciones viables:

| $\{x_0, x_1, x_2\}$                                      | $\sum w_i x_i$ | $\sum b_i x_i$ |
|----------------------------------------------------------|----------------|----------------|
| ★ $\left\{\frac{1}{2}, \frac{1}{3}, \frac{1}{4}\right\}$ | 16,5           | 24,25          |
| ★ $\left\{1, \frac{2}{15}, 0\right\}$                    | 20             | 28,2           |
| ★ $\left\{0, \frac{2}{3}, 1\right\}$                     | 20             | 31             |

### ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .
- ▶ Soluciones viables:

| $\{\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2\}$           | $\sum \mathbf{w}_i \mathbf{x}_i$ | $\sum \mathbf{b}_i \mathbf{x}_i$ |
|----------------------------------------------------------|----------------------------------|----------------------------------|
| ★ $\left\{\frac{1}{2}, \frac{1}{3}, \frac{1}{4}\right\}$ | 16,5                             | 24,25                            |
| ★ $\left\{1, \frac{2}{15}, 0\right\}$                    | 20                               | 28,2                             |
| ★ $\left\{0, \frac{2}{3}, 1\right\}$                     | 20                               | 31                               |
| ★ $\left\{0, 1, \frac{1}{2}\right\}$                     | 20                               | 31,5                             |

- Ejemplo: (cont.)

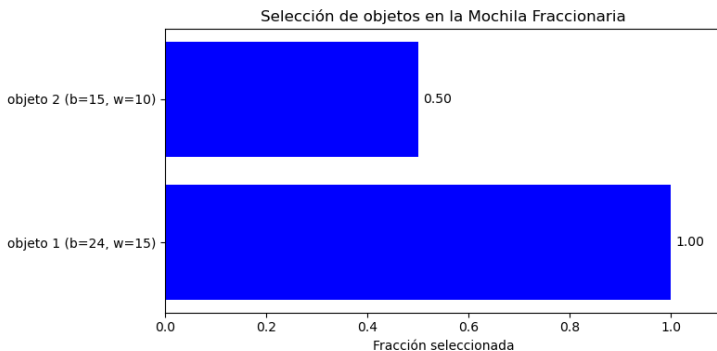


Figura 6: Solución viable: 31,5

# Algoritmo voraz: Mochila fraccional

```
1: function MOCHILA FRACCIONAL(objetos, capacidad)
2: Input: objetos; capacidad; peso
3: Output: BeneficioTotal.
4:   Ordenar objetos por (beneficio / peso) de mayor a menor
5:   BeneficioTotal:= 0
6:   for Para cada (Beneficio, peso) en objetos do
7:     if capacidad de mochila es 0 then
8:       Salir
9:     end if
10:    if peso  $\leq$  capacidad then
11:      BeneficioTotal := BeneficioTotal + Beneficio
12:      capacidad := capacidad - peso
13:    else
14:      fracción:= capacidad / peso
15:      BeneficioTotal:= BeneficioTotal + (Beneficio  $\times$  fracción)
16:      capacidad := 0
17:    end if
18:  end for
19:  Retornar BeneficioTotal
20: end function
```

- Ejemplo:

- Ejemplo:
  - ▶ Objetos:  $n = 3$ .

- Ejemplo:

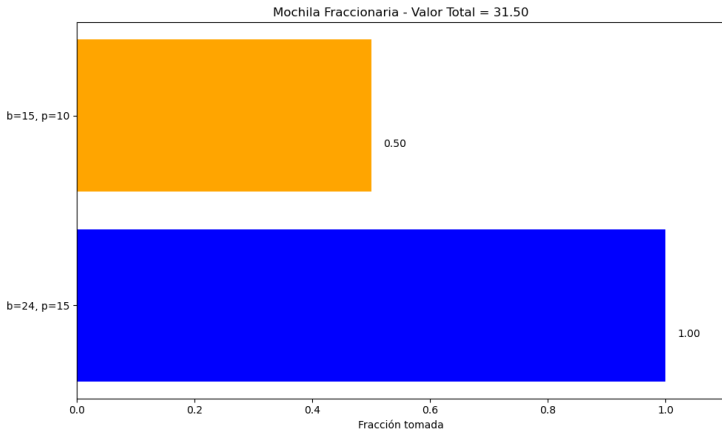
- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{18, 15, 10\}$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_1, b_2, b_3\} = \{25, 24, 15\}$ .



# Modelo de optimización: Mochila binaria

- Modelo:

$$\text{MAX. } Z = \sum_{1 \leq i \leq n} b_i x_i \quad \text{Beneficio o utilidad} \quad (4)$$

s.a.

$$\sum_{1 \leq i \leq n} w_i x_i \leq m \quad \text{Llenar mochila según su capacidad } m \quad (5)$$

$$x_i \in \{0, 1\} \text{ binario} \quad (6)$$

- Ejemplo:

- Ejemplo:
  - ▶ Objetos:  $n = 3$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .

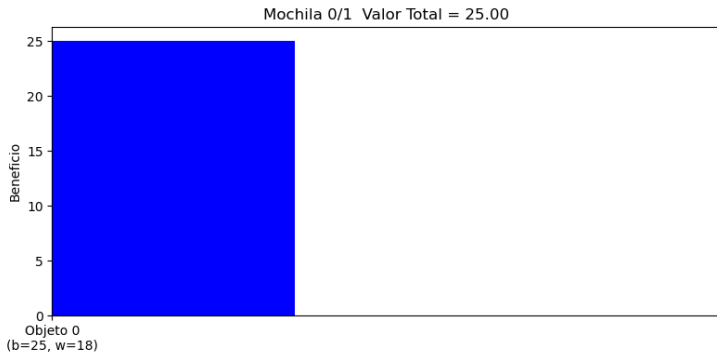
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_0, w_1, w_2\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_0, b_1, b_2\} = \{25, 24, 15\}$ .



## Algoritmo voraz: Mochila binaria

- El AG no garantiza solución óptima para la mochila binaria (0-1).

## Algoritmo voraz: Mochila binaria

- El AG no garantiza solución óptima para la mochila binaria (0-1).
- En la mochila 0-1, solo existen dos posibilidades, es decir, se puede escoger un objeto completo o no se lo escoge; por ello el AG puede dar soluciones subóptimas.

## Algoritmo voraz: Mochila binaria

- El AG no garantiza solución óptima para la mochila binaria (0-1).
- En la mochila 0-1, solo existen dos posibilidades, es decir, se puede escoger un objeto completo o no se lo escoge; por ello el AG puede dar soluciones subóptimas.
- **Observación:** El AG solo funciona de manera óptima para el caso de la mochila fraccionaria, pues es posible considerar fracciones de los objetos a ser incluidos en la mochila según su capacidad.

## ● Seudocódigo

```
1: function MOCHILA BINARIA(objetos, capacidad)
2: Input: objetos; capacidad; peso; beneficio
3: Output: BeneficioTotal; ObjetosSeleccionados.
4:   Ordenar objetos por (beneficio / peso) de mayor a menor
5:   BeneficioTotal:= 0
6:   ObjetosSeleccionados :=  $\phi$ 
7:   for Cada (beneficio, peso) en objetos do
8:     if peso  $\leq$  capacidad then
9:       Agregar (beneficio, peso) a los ObjetosSeleccionados
10:      BeneficioTotal := BeneficioTotal + beneficio
11:      capacidad := capacidad - peso
12:    end if
13:  end for
14:  Retornar BeneficioTotal, ObjetosSeleccionados
15: end function
```

- Ejemplo:

- Ejemplo:
  - ▶ Objetos:  $n = 3$ .

- Ejemplo:

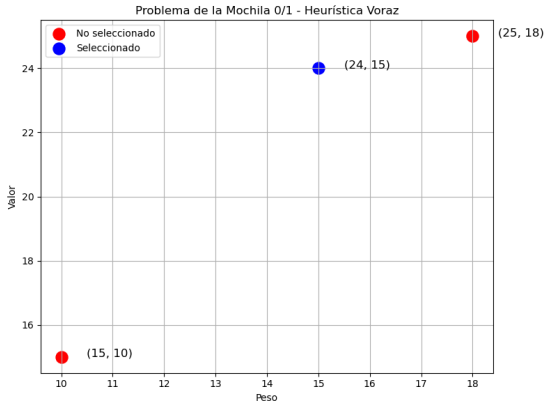
- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{18, 15, 10\}$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{18, 15, 10\}$ .
- ▶ Capacidad (kg.):  $m = 20$ .
- ▶ Beneficio (s/.):  $b = \{b_1, b_2, b_3\} = \{25, 24, 15\}$ .



- Ejemplo:

- Ejemplo:
  - ▶ Objetos:  $n = 3$ .

- Ejemplo:

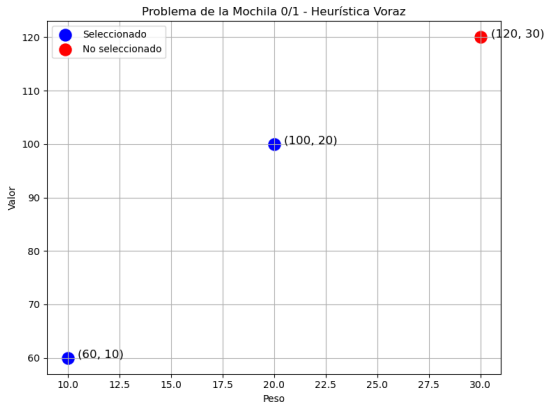
- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{10, 20, 30\}$ .

- Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{10, 20, 30\}$ .
- ▶ Capacidad (kg.):  $m = 50$ .

## ● Ejemplo:

- ▶ Objetos:  $n = 3$ .
- ▶ Peso del objeto (kg.):  $w = \{w_1, w_2, w_3\} = \{10, 20, 30\}$ .
- ▶ Capacidad (kg.):  $m = 50$ .
- ▶ Beneficio (s/.):  $b = \{b_1, b_2, b_3\} = \{60, 100, 120\}$ .



- Propiedades:

- **Propiedades:**

- ▶ **Lema 1:** Si la suma de todos los pesos es  $\leq m$ , entonces  $x_i = 1$ , donde  $1 \leq i \leq n$ , es una solución óptima.

- **Propiedades:**

- ▶ **Lema 1:** Si la suma de todos los pesos es  $\leq m$ , entonces  $x_i = 1$ , donde  $1 \leq i \leq n$ , es una solución óptima.
- ▶ **Lema 2:** Todas las soluciones óptimas llenarán exactamente la mochila.

El Lema 2 es verdadero pues siempre se puede incrementar la contribución de algun objeto  $i$  por una cantidad fraccional hasta que el peso total sea exactamente  $m$ .

# Aplicaciones del problema de la mochila

- Planificación de producción y logística:

# Aplicaciones del problema de la mochila

- Planificación de producción y logística:

- ▶ Selección de productos para envío: Escoger qué productos se deben incluir en un camión o contenedor, de modo que se maximice el valor sin exceder el peso o volumen.

# Aplicaciones del problema de la mochila

- Planificación de producción y logística:

- ▶ Selección de productos para envío: Escoger qué productos se deben incluir en un camión o contenedor, de modo que se maximice el valor sin exceder el peso o volumen.
- ▶ Empaquetado inteligente: Decidir qué artículos incluir en un paquete considerando las limitaciones de espacio y valor para disminuir los costos de transporte.

# Aplicaciones del problema de la mochila

- Planificación de producción y logística:

- ▶ Selección de productos para envío: Escoger qué productos se deben incluir en un camión o contenedor, de modo que se maximice el valor sin exceder el peso o volumen.
- ▶ Empaquetado inteligente: Decidir qué artículos incluir en un paquete considerando las limitaciones de espacio y valor para disminuir los costos de transporte.

- Asignación de recursos en computación:

# Aplicaciones del problema de la mochila

- **Planificación de producción y logística:**

- ▶ **Selección de productos para envío:** Escoger qué productos se deben incluir en un camión o contenedor, de modo que se maximice el valor sin exceder el peso o volumen.
- ▶ **Empaquetado inteligente:** Decidir qué artículos incluir en un paquete considerando las limitaciones de espacio y valor para disminuir los costos de transporte.

- **Asignación de recursos en computación:**

- ▶ **Carga en servidores:** Asignar tareas a los servidores con recursos limitados (CPU, memoria) maximizando el uso eficiente.

# Aplicaciones del problema de la mochila

- **Planificación de producción y logística:**

- ▶ **Selección de productos para envío:** Escoger qué productos se deben incluir en un camión o contenedor, de modo que se maximice el valor sin exceder el peso o volumen.
- ▶ **Empaquetado inteligente:** Decidir qué artículos incluir en un paquete considerando las limitaciones de espacio y valor para disminuir los costos de transporte.

- **Asignación de recursos en computación:**

- ▶ **Carga en servidores:** Asignar tareas a los servidores con recursos limitados (CPU, memoria) maximizando el uso eficiente.
- ▶ **Sistemas embebidos o IoT:** Seleccionar procesos o módulos a ejecutar dentro de límites de energía o procesamiento.

- Telecomunicaciones y redes:

- **Telecomunicaciones y redes:**

- ▶ **Asignación de canales o ancho de banda:** Seleccionar qué conexiones se deben priorizar para maximizar el rendimiento con un número limitado de canales.

- Telecomunicaciones y redes:

- ▶ Asignación de canales o ancho de banda: Seleccionar qué conexiones se deben priorizar para maximizar el rendimiento con un número limitado de canales.

- Comercio electrónico y recomendaciones:

- Telecomunicaciones y redes:

- ▶ Asignación de canales o ancho de banda: Seleccionar qué conexiones se deben priorizar para maximizar el rendimiento con un número limitado de canales.

- Comercio electrónico y recomendaciones:

- ▶ Recomendación de productos: Sugerir productos a un usuario bajo la restricción de presupuesto, maximizando el interés o valor estimado para el cliente.

- Telecomunicaciones y redes:

- ▶ Asignación de canales o ancho de banda: Seleccionar qué conexiones se deben priorizar para maximizar el rendimiento con un número limitado de canales.

- Comercio electrónico y recomendaciones:

- ▶ Recomendación de productos: Sugerir productos a un usuario bajo la restricción de presupuesto, maximizando el interés o valor estimado para el cliente.

- Toma de decisiones bajo restricciones:

- **Telecomunicaciones y redes:**

- ▶ **Asignación de canales o ancho de banda:** Seleccionar qué conexiones se deben priorizar para maximizar el rendimiento con un número limitado de canales.

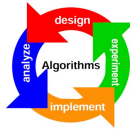
- **Comercio electrónico y recomendaciones:**

- ▶ **Recomendación de productos:** Sugerir productos a un usuario bajo la restricción de presupuesto, maximizando el interés o valor estimado para el cliente.

- **Toma de decisiones bajo restricciones:**

- ▶ **Planificación estratégica:** Elegir iniciativas o tareas a priorizar dentro de límites de tiempo, personal o presupuesto, para de este modo maximizar el beneficio.

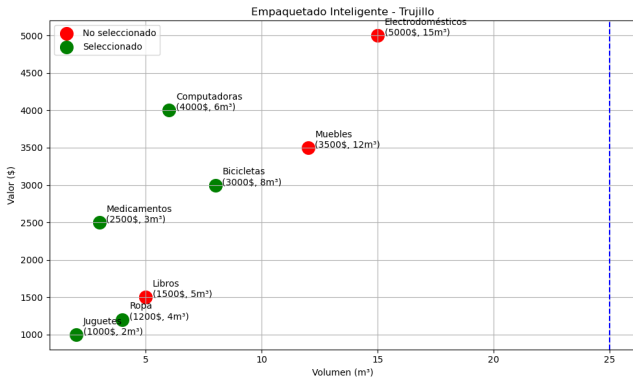
# Estrategia algorítmica voraz



- 1 Algoritmo voraz
- 2 Relación entre algoritmo voraz y optimización
- 3 Problema de la mochila
  - Modelo de optimización: Mochila fraccional
  - Algoritmo voraz: Mochila fraccional
  - Modelo de optimización: Mochila binaria
  - Algoritmo voraz: Mochila binaria
  - Aplicaciones del problema de la mochila
- 4 Mochila para empaquetado inteligente

# Mochila para empaquetado inteligente

- Ejemplo 1:** Cada artículo que tiene un valor y ocupa un espacio será colocado en un camión o contenedor. Por lo tanto, se desea maximizar el valor total de lo que se transportará.



- **Ejemplo 2:** Cada zona de un Distrito tiene espacio para colocar un grupo de artículos. Se desea maximizar el valor total de lo almacenado.

